ModalDish: finite-element modal synthesis of plates and membranes of arbitrary shape

Technical documentation

Olivier Doaré*

July 2026 — ModalDish v0.1

Abstract

ModalDish is an audio plugin that simulates the linear and weakly nonlinear vibration of a thin plate (or tensioned membrane) of arbitrary user-drawn shape, with mixed boundary conditions. The plate eigenmodes are computed by a non-conforming (Morley) finite-element discretisation and a dense subspace eigensolver, both implemented in the reusable, JUCE-free library `FxmeTools/acoustics`. Sound is produced by a bank of resonant band-pass filters, one per mode, following the modal-synthesis approach of the MechanOdd project. A first-order tension law allows audio-rate retuning of the whole bank without re-solving the eigenproblem, and this same mechanism supports an efficient approximation of geometric (von Kármán–Berger) nonlinearity: amplitude-dependent pitch glide and a cubic inter-mode energy-cascade surrogate, both realtime-safe and unconditionally stable. This document details the mathematical model, the discretisation, the eigensolution, the audio implementation and the nonlinear approximations, with references to the literature.

Contents

*FX-Mechanics, `github.com/odoare`. Model, implementation and documentation developed with Claude (Anthropic).

1 Continuous model

1.1 Scaled Kirchhoff plate with tension

The transverse deflection $w(\boldsymbol{x}, t)$ of a thin plate occupying a domain $\Omega \subset \mathbb{R}^2$ is modelled by Kirchhoff–Love theory [1, 2] augmented by a uniform in-plane tension N and two damping mechanisms:

$$\rho h\, \ddot{w} \;+\; c_1 \dot{w} \;+\; c_2\, \Delta^2 \dot{w} \;+\; D\, \Delta^2 w \;-\; N\, \Delta w \;=\; f(\boldsymbol{x}, t), \qquad D = \frac{E h^3}{12(1-\nu^2)}, \tag{1}$$

where ρh is the surface density, D the flexural rigidity, ν the Poisson ratio ($\nu = 0.3$ in ModalD-ish), c_1 a viscous (air/support) damping coefficient and c_2 a stiffness-proportional (material, Kelvin–Voigt-type) damping coefficient. Dividing by ρh and rescaling space and time so that the coefficients of $\ddot{w}$ and $\Delta^2 w$ are unity yields the *scaled* equation actually used throughout the code:

$$\ddot{w} + c_v\, \dot{w} + c_m\, \Delta^2 \dot{w} + \Delta^2 w - T\, \Delta w = f, \tag{2}$$

with a single dimensionless *tension-to-flexural-stiffness* parameter $T \geq 0$. $T = 0$ is a pure plate; $T \to \infty$ approaches the membrane limit. Frequencies obtained from (2) are dimensionless; the plugin maps the first eigenfrequency onto the user parameter f_1 and all others follow proportionally (§4), so the absolute material constants never need to be specified.

1.2 Boundary conditions

The boundary $\partial\Omega$ is split by the user into segments, each carrying one of four classical conditions:

Name	Essential constraints	Natural conditions	DOFs constrained
Free	—	$M_n = 0$, $V_n = 0$	—
Simple support	$w = 0$	$M_n = 0$	vertex w
Clamp	$w = 0$, $\partial_n w = 0$	—	vertex w, edge $\partial_n w$
Sliding support	$\partial_n w = 0$	$V_n = 0$	edge $\partial_n w$

where M_n and V_n are the bending moment and effective (Kirchhoff) shear. Only the *essential* conditions are enforced; the natural ones are satisfied weakly by the variational formulation [3]. The last column anticipates §2.2: the four conditions map one-to-one onto the degrees of freedom of the Morley element, which is the main reason this element was chosen.

2 Finite-element discretisation

2.1 Mesh generation

The library meshes an arbitrary closed polygon (the user's outline, sampled from a closed Catmull–Rom spline [4] for freehand shapes) with a target element size h:

1. *Boundary resampling.* The outline is walked at arc-length spacing $\approx h$. Vertices where the outline turns by more than 30° are detected as corners and kept exactly, so rectangles remain rectangles. Every boundary sample stores its arc-length parameter $t \in [0, 1)$, which is later used to attach the per-segment boundary conditions.

2. *Interior lattice.* A hexagonal lattice of pitch h fills the bounding box; points closer than $0.62\,h$ to the boundary or outside the polygon are discarded. A small deterministic jitter ($\pm 0.06\,h$) breaks the cocircular degeneracies that regular lattices would otherwise present to the triangulator.

3. *Triangulation.* Bowyer–Watson incremental Delaunay insertion [5, 6], with a super-triangle and cavity re-triangulation. The implementation is the plain $O(n^2)$ variant, entirely adequate for the $n \lesssim 2000$ points used here.

4. *Clipping.* Triangles whose centroid falls outside the polygon are removed (this handles non-convex outlines), orientation is normalised to counter-clockwise, and an edge table with adjacency is built. Edges with a single adjacent triangle are boundary edges; they inherit the wrap-aware midpoint of their endpoints' arc parameters.

2.2 The Morley element

The biharmonic operator requires C^1 continuity for conforming elements, which leads to high-order elements (Argyris, Bell) that are expensive and delicate. ModalDish instead uses the *Morley triangle* [7], the simplest non-conforming plate bending element: on each triangle the deflection is a full quadratic polynomial (6 coefficients) determined by 6 degrees of freedom,

- the deflections w_i at the three vertices,
- the normal derivatives $(\partial_n w)_e$ at the three edge midpoints.

The element is non-conforming (w is discontinuous along edges except at vertices), yet it converges for fourth-order problems, and its eigenvalue approximations converge at rate $O(h^2)$, *from below* [8, 9] — both properties are verified numerically in §2.4. Crucially for ModalDish, its two DOF families are exactly the quantities constrained by the four boundary conditions of §1.2.

Rather than hard-coding the classical closed-form shape functions, each element inverts a 6×6 generalized Vandermonde system: writing the local polynomial in centroid-centred monomials $p(\boldsymbol{x}) = \sum_m c_m\, \mu_m(\boldsymbol{x})$, $\{\mu_m\} = \{1, X, Y, X^2, XY, Y^2\}$, the matrix $C_{im} = \ell_i(\mu_m)$ of the six DOF functionals ℓ_i is inverted by Gaussian elimination with partial pivoting; column j of C^{-1} holds the monomial coefficients of shape function N_j. Centroid-centred coordinates keep C well-conditioned at the element scale. The mid-edge normal is the *global* normal of the edge (stored with $v_0 < v_1$, normal to the left of $v_0 \to v_1$), so the shared DOF has the same meaning in both adjacent elements.

2.3 Element matrices

With constant curvatures per element (second derivatives of a quadratic), the bending stiffness is exact:

$$K^e_{ij} = A_e\, \boldsymbol{\kappa}_i^{\mathsf{T}} \mathbf{D}\, \boldsymbol{\kappa}_j, \qquad \boldsymbol{\kappa}_j = \begin{pmatrix} \partial_{xx} N_j \\ \partial_{yy} N_j \\ 2\,\partial_{xy} N_j \end{pmatrix}, \qquad \mathbf{D} = \begin{pmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{pmatrix}, \tag{3}$$

(A_e = element area; flexural rigidity is 1 after scaling). The tension (geometric) matrix and the consistent mass matrix are integrated with quadrature rules exact for their integrands:

$$G^e_{ij} = \int_e \nabla N_i \cdot \nabla N_j \, \mathrm{d}A \qquad \text{3-point midpoint rule (degree 2, exact),} \tag{4}$$

$$M^e_{ij} = \int_e N_i N_j \, \mathrm{d}A \qquad \text{6-point degree-4 rule [12] (quartic, exact).} \tag{5}$$

G is the discrete counterpart of $-\Delta$ restricted to the plate space and doubles as the *geometric stiffness* familiar from buckling analysis [3]. Global assembly scatters K^e, G^e, M^e into dense symmetric matrices over the *free* DOFs only: essential boundary conditions (§1.2) are applied by elimination, using each boundary vertex's / edge's arc parameter to look up its segment's condition. At a junction between two segments the vertex takes the strongest condition (clamp > simple support > sliding > free).

2.4 Validation

Three console suites are built alongside the plugin: `Tests/FemTests.cpp` for the finite-element core, `Tests/IoTests.cpp` for the audio path (pickup mixing, tail stereo placement, note handling, sample-rate independence) and `Tests/CascadeMeasure.cpp` for the nonlinear voicing (§5.2). The first verifies the implementation against classical analytical results [1]:

- simply supported unit square: $\lambda_{mn} = \pi^4(m^2+n^2)^2$, with the measured $O(h^2)$ convergence from below (error ratio 3.9 when h is halved) and the exact tension coefficient $g_{11} = 2\pi^2$ (§3.2);
- clamped circle: frequency ratios 2.081, 3.414, 3.893 and the absolute fundamental $\omega_{01} = (\beta_{01}/R)^2$, $\beta^2_{01} = 10.2158$ [1], within 2–5% at moderate mesh density;
- mixed clamped/free boundaries, mode-shape vanishing on the clamped side, and the first-order tension law against an exact re-solve.

3 The eigenvalue problem

3.1 Formulation and solver

Discretisation of (2) (undamped part) yields the symmetric generalized eigenproblem

$$(K + T_0\, G)\, \boldsymbol{\varphi}_k \;=\; \lambda_k\, M \boldsymbol{\varphi}_k, \qquad \lambda_k = \omega_k^2, \qquad \boldsymbol{\varphi}_k^{\mathsf{T}} M \boldsymbol{\varphi}_l = \delta_{kl}, \tag{6}$$

solved at a *reference tension* T_0 (the value of the Tension knob when the user presses *Compute*). With n free DOFs ($n \approx$ vertices + edges $\approx 4\times$ vertices, typically 300–5000 over the Grid range) and only the lowest $m \le 256$ modes wanted (capped at $\sim n/6$ so the top modes stay resolved by the mesh), the solver is classical *subspace iteration with Rayleigh–Ritz projection* [13, 3]:

1. Form $A = K + T_0 G$ and the shifted operator $P = A + \sigma M$ with $\sigma = 10^{-5}\,\mathrm{tr}(A)/\mathrm{tr}(M)$. The shift keeps P positive definite when free (Neumann-type) boundaries leave rigid-body modes in A's null space. P is Cholesky-factored once ($O(n^3/3)$).
2. Iterate on a block of $p = m + \max(8, m/2)$ vectors: $Z \leftarrow P^{-1}MX$ (one inverse-power step towards the low spectrum), followed by Rayleigh–Ritz on $\mathrm{span}(Z)$: project $A_p = Z^{\mathsf{T}}AZ$, $M_p = Z^{\mathsf{T}}MZ$, solve the $p \times p$ generalized problem by Cholesky reduction and cyclic Jacobi [14], and take the Ritz vectors as the next block X.

3. Stop when the wanted eigenvalues are stationary: relative 10^{-6} on the lower half, 10^{-4} on the upper half — the slowest to converge, and the modes whose FEM discretisation error is orders of magnitude larger anyway (both thresholds are far below one cent in frequency); or after 60 iterations.

Implementation notes that matter for speed: the iteration block is stored *transposed* (every iteration vector contiguous in memory, so all $O(n^2 p)$ loops stream cache lines), the independent per-vector solves and matrix-vector products are spread over a few worker threads, and the Jacobi solver of the projected $p \times p$ problem uses a *relative* off-diagonal convergence test (the entries scale with $\lambda^2 \sim 10^{12}$; an absolute threshold silently burns the full sweep budget on every call). Together these give a $\sim 4\times$ speed-up over the straightforward implementation (23.6 s $\to$ 5.6 s for 128 modes on a 931-DOF mesh).

Sparse storage. A Morley element couples a degree of freedom only to the five others on its own triangle, so an assembled row of K, G or M holds about eleven non-zeros — and, this being the useful part, about eleven whatever the mesh density. Dense storage of the same matrix is n per row, so the fraction it wastes grows without bound: at $n = 5000$ the three matrices are 24 million doubles standing in for 55 thousand real numbers. They are therefore assembled in compressed sparse rows, in two passes over the triangles: the first gathers nothing but the six DOF indices of each element and builds the sparsity pattern (one pattern, shared by all three matrices — same mesh, same couplings), the second is the expensive one and scatters element values into it. Both passes read the DOF indices through the same helper, so the pattern cannot fail to cover what the assembly scatters.

Renumbering, and the factorisation. Sparse operators alone are not enough, because Cholesky fills in: L is non-zero wherever the original matrix has a non-zero anywhere to its left in the same row, which for a mesh numbered as the generator happened to produce it means very nearly everywhere. The factor of a sparse matrix in a careless order is a dense matrix.

What fixes this is not a cleverer factorisation but a better numbering. Renumbering the unknowns so that coupled ones sit close together traps the fill inside a narrow envelope around the diagonal, and everything strictly outside that envelope is then exactly zero — in floating point as much as in exact arithmetic — so a factorisation that stores only the envelope is the complete factor with the provably-zero part left out. The renumbering used is reverse Cuthill–McKee [10]: a breadth-first walk of the coupling graph that visits each frontier in order of increasing degree, started from a pseudo-peripheral node (George–Liu [11]), then reversed — which leaves the bandwidth unchanged but strictly shrinks the profile, the quantity that is actually paid for. On these meshes it takes the mean row bandwidth from roughly $n/1.5$ to $1.11\sqrt{n}$, a fit good to a few percent from $n = 554$ to $n = 19700$; at $n = 4913$, from 3681 to 197.

The permutation is applied in the *degree-of-freedom map*, before anything is assembled, and that is the whole of it. No matrix is permuted, no eigenvector is permuted back, and the vertex values exported at the end are read through the same renumbered map they were assembled through. Applying it anywhere closer to the solver would mean threading it through each of those steps separately, which is precisely the change that yields plausible-looking wrong mode shapes.

What it costs now. Nothing in the solve is quadratic in n any more:

$$\underbrace{32\,p\,n}_{\text{iteration block}} \;+\; \underbrace{8.9\,n^{3/2}}_{\text{profile factor}} \;+\; \underbrace{\mathcal{O}(n)}_{\text{sparse operators}} \quad \text{bytes,} \qquad \text{against } 32n^2 \text{ dense,} \tag{7}$$

and the leading term is the iteration block, 4 vectors of $p \times n$ for a block of $p = m + \max(8, m/2)$. That is worth stating plainly: what a solve costs is now set by *how many modes are asked for*,

and only linearly by how fine the mesh is. Measured on the same machine, same meshes, all three paths run back to back:

		dense		sparse + profile		
n	modes	time	memory	time	memory	gain
2149	64	13.1 s	125 MB	0.45 s	7 MB	29× / 17×
3841	128	114.8 s	419 MB	3.6 s	25 MB	32× / 17×
6029	128	335.5 s	1040 MB	5.5 s	39 MB	61× / 26×
8577	256	—	2.4 GB	35.3 s	107 MB	—
15381	256	—	7.6 GB	48.4 s	196 MB	—
23865	256	—	18.2 GB	83.8 s	309 MB	—

The last two rows have no dense column because the dense solver cannot run them at all; the figure given is what it would have had to allocate. The gain is larger than the $O(n^3/3)$ factorisation alone accounts for, because the factorisation was never the expensive part: the p triangular solves *per iteration* were, and they fall from $O(n^2)$ to $O(n^{3/2})$ along with it.

Accuracy is unaffected where it is defined to be. Changing the elimination order changes the rounding, so the sparse and dense paths no longer agree bit for bit; they agree to the extent the iteration was asked to converge, and the split is sharp. On a 24-mode solve the first twelve — held to 10^{-6} — agree to 10^{-14} in λ and 4×10^{-6} in shape, while the last few, entitled only to 10^{-4}, differ by 10^{-5}. The test suite checks the two halves separately, since a single loose tolerance would pass even if the converged modes had quietly gone wrong.

The remaining ceiling is time, not memory, and it is a ceiling on the *mode count*: the $O(np^2)$ Rayleigh–Ritz projections and the $4pn$ block are what a larger solve now pays for. The editor shows the projected footprint next to the mesh size once it passes 48 MB.

Code layout. None of the above is about plates, and it is not written as though it were. The matrix storage, the Cholesky and Jacobi kernels and the subspace iteration live in `FxmeTools/math`; `FxmeTools/acoustics` keeps the boundary conditions, the Morley element and the modal quantities. The eigensolver never sees a matrix at all — it multiplies through a `SymmetricOperator` and solves through an `SpdSolver` — which is what makes the storage a choice at the call site (`ModalOptions::storage`, sparse by default, dense kept as the reference the sparse path is validated against) rather than a rewrite, and what will let a profile factorisation drop in as one more `SpdSolver` with nothing else moving.

Finally, the computed modal data (eigenvalues, tension coefficients, shapes, statistical tail) is cached inside the plugin state: loading a session or preset republishes the model immediately, with no recomputation. The mesh is not serialised — `generateMesh` is a pure deterministic function of the outline and density, so it is rebuilt on load and checked against the stored vertex count. Eigenvalues below an absolute threshold (10^{-4}; rigid-body and near-rigid modes of free plates) are dropped. Eigenvectors are mass-normalised, their vertex values are exported for display and for the pointwise evaluations $\varphi_k(\boldsymbol{x})$ used by the synthesis (linear interpolation on the located triangle; the mid-edge DOFs remain internal). The whole computation runs on a background thread with progress reporting; completed models are published to the audio thread by an atomic pointer swap with generation-based reclamation.

3.2 Audio-rate tension changes without re-solving

The solver additionally returns, for every mode, the *tension sensitivity*

$$g_k \;=\; \boldsymbol{\varphi}_k^{\mathsf{T}} G\, \boldsymbol{\varphi}_k \quad \left(= \int_\Omega |\nabla \varphi_k|^2 \mathrm{d}A \right), \tag{8}$$

enabling the first-order frequency law used at audio rate:

$$\boxed{\omega_k^2(T) \;=\; \lambda_k + (T - T_0)\, g_k} \tag{9}$$

This is legitimate because T enters the operator *linearly* and the Rayleigh quotient is *stationary* at eigenvectors: perturbing T changes $\boldsymbol{\varphi}_k$ at first order in $(T - T_0)$, but that eigenvector error affects the eigenvalue only at *second* order. Equation (9) is the standard eigenvalue derivative $\partial\lambda_k/\partial T = \boldsymbol{\varphi}_k^{\mathsf{T}} G \boldsymbol{\varphi}_k$ of structural sensitivity analysis [17], a special case of classical perturbation theory [15, 16]; here it is exact at T_0 with $O\big((T - T_0)^2\big)$ error. For the simply supported square the law is exact for all T (the eigenvectors do not depend on T); in general, pressing *Compute* re-anchors T_0 when the knob has travelled far. The validation suite checks (9) against an exact re-solve at $T = 50$ (error $< 1\%$).

Beyond the knob itself, (9) is the enabling mechanism for the nonlinear glide of §5.1: *dynamic* tension is just a fast-moving T.

4 Modal synthesis with resonant filter banks

4.1 Modal decomposition

Expanding $w(\boldsymbol{x}, t) = \sum_k q_k(t)\, \varphi_k(\boldsymbol{x})$ in (2) and using M-orthonormality decouples the dynamics into damped oscillators

$$\ddot{q}_k + 2\zeta_k \omega_k\, \dot{q}_k + \omega_k^2 q_k = \varphi_k(\boldsymbol{x}_h)\, F(t), \qquad w(\boldsymbol{x}_o, t) = \sum_k \varphi_k(\boldsymbol{x}_o)\, q_k(t), \tag{10}$$

for a point force F at $\boldsymbol{x}_h$ and a pickup at $\boldsymbol{x}_o$ — the standard modal synthesis framework [18, 19, 20, 21]. ModalDish realises each oscillator as a second-order IIR resonator, exactly as in the MechanOdd modal resonators: one constant-peak-gain band-pass biquad per mode (RBJ "cookbook" coefficients [22]) with $Q_k = 1/(2\zeta_k)$, excited by $\varphi_k(\boldsymbol{x}_h)F$ and weighted on output by $\varphi_k(\boldsymbol{x}_o)$. Up to 1024 modes are active — though the bank stops at the highest one still below $0.47 f_s$, which at a high f_1 is a small fraction of them; the rest would cost a resonator each and return silence. The bank is allocation-free and retunable on the audio thread.

4.2 Frequency and damping laws

The user selects the fundamental in hertz and the bank plays at

$$f_k \;=\; f_1\, \frac{\omega_k(T_{\text{eff}})}{\omega_1(T_{\text{knob}})}, \qquad T_{\text{eff}} = T_{\text{knob}} + T_{\text{dyn}}, \tag{11}$$

with all ω_k given by (9). The normalisation reference matters: it is the fundamental at the *static* knob tension, not the instantaneous one. The static Tension knob therefore keeps mode 1 pinned at f_1 and only reshapes the overtone ratios (the timbre morphs from plate towards membrane), whereas the *dynamic* tension of §5.1 shifts the whole spectrum, fundamental included — the audible hardening glide. Normalising by the instantaneous fundamental instead would cancel the glide of mode 1 and leave only the overtone-ratio compression towards the membrane spectrum (g_k/g_1 ratios are lower than the plate ratios $\sqrt{\lambda_k/\lambda_1}$), which is perceived as a *softening* — the opposite of the physical behaviour. So the timbre (inharmonicity pattern) is set by geometry, boundary conditions and tension, while f_1 transposes it. Modes falling outside $[20\,\text{Hz},\ 0.47 f_s]$ are muted.

Projecting the two damping terms of (2) onto mode k gives modal damping ratios (using $\Delta^2 \varphi_k \approx \omega_k^2 \varphi_k$ for the material term)

$$\zeta_k \;=\; \underbrace{\frac{c_v}{2\omega_k}}_{\text{viscous}} + \underbrace{\frac{c_m\, \omega_k}{2}}_{\text{material}} \;=\; \frac{\zeta_v}{\nu_k} + \zeta_m\, \nu_k, \tag{12}$$

where the knobs ζ_v, ζ_m are simply the two contributions to the damping ratio *of mode 1*. Viscous damping therefore lets high modes ring relatively longer (constant absolute bandwidth), material damping kills them faster — the two classical, perceptually distinct decay profiles of struck plates [23, 24].

4.3 Excitation, pickup, gain compensation

A strike at $\boldsymbol{x}_h$ — from the mouse, or from a MIDI note (§4.4) — launches a half-sine force pulse (a standard hammer/impact model [23])

$$F(t) = F_0 \sin(\pi t/\tau)\,, \qquad 0 \le t \le \tau, \tag{13}$$

with duration τ (*Hammer* knob, 0.1–50 ms: short = bright, long = mellow, since the pulse spectrum rolls off above $\approx 1/\tau$) and amplitude F_0 (*Force* knob × strike velocity).

Up to 16 pulses may be in contact at once, each carrying its own weight vector $\varphi_k(\boldsymbol{x}_h)$, and all of them are summed into the drive of a *single* bank. There is one plate, struck several times, not several plates: a second strike adds to the ringing of the first and shares its decay, as a real gong does, and no note has an envelope or a release of its own. The slot count therefore sizes simultaneous *contact*, which lasts milliseconds, not sustain; slots are reused round-robin, and reuse can only truncate a pulse still in contact, never a ringing tail, since the tail lives in the shared bank.

The plate is listened to at up to 8 *pickups* and struck at up to 8 *sources*, each an automatable point: a pickup carries position, level, pan and an on switch, a source position, hammer duration, force, positional spread, input send, input balance, MIDI note and an on switch. Both collapse before the audio loop — the pickups into one pair of per-mode output weights (§5.4), the sources into one pair of per-mode input weights — so the per-sample cost is that of a single point however many are switched on, and a point that is off costs not even its mode-shape evaluation. The external audio input reaches the plate through the sources' sends (*effect mode*), each source taking its own balance of the incoming stereo pair.

The collapse is what makes per-pickup metering awkward, since after it no individual pickup's signal exists anywhere. A pickup's panel therefore carries a third weight vector, $c_k\,\ell_p\,\varphi_k(\boldsymbol{x}_p)$: mono, with the pickup's level ℓ_p but not its pan, so that it reads what the point is hearing rather than where that lands in the image. Only the pickup whose panel is open is metered — one extra multiply-add per mode while a panel is up, against the eight that metering all of them would cost, which is four times the stereo mix itself.

All the meters read peak, held and falling at 20 dB/s, rather than RMS. The quantity they are read for is clipping headroom, and a struck plate is peaky: measured on the default plate its 100 ms RMS sits 7.7 to 11.6 dB below its sample peak across hammer times and dampings, so an RMS bar reads +3 dBFS on an output that is really at +13. The hold is accumulated per block on the audio thread, so a transient cannot fall between two GUI frames; polled at 30 Hz it tracks the true peak to better than 0.7 dB.

Output gain, and why it is exactly $1/\zeta_k$. The RBJ constant-0 dB-peak band-pass has, as an analog prototype,

$$H_k(s) \;=\; \frac{2\zeta_k\omega_k\, s}{s^2 + 2\zeta_k\omega_k s + \omega_k^2}, \tag{14}$$

which is $2\zeta_k\omega_k$ times the *velocity* transfer function of the modal oscillator. The bank therefore does not output the plate's motion but its motion scaled by each mode's own bandwidth, and an impulse response whose peak ought to be fixed by the strike alone peaks proportionally to ζ_k instead. Damping would set the attack level as well as the decay — the opposite of the physics, where a struck mode's amplitude is set by the impulse and damping can only take energy away afterwards.

Dividing that factor straight back out is the whole of the correction. With

$$c_k \;=\; \zeta^{c}_{\mathrm{ref}}/\zeta_k, \qquad \zeta^{c}_{\mathrm{ref}} = 10^{-3}, \tag{15}$$

the chain becomes $c_k H_k(s) = 2\zeta^{c}_{\mathrm{ref}}\omega_k\, s/(s^2 + \ldots)$: an exact model of modal *acceleration*, up to the constant $2\zeta^{c}_{\mathrm{ref}}$ — which is the right output quantity in any case, a plate radiating pressure proportional to its acceleration. Strike and damping then stop interfering,

$$\text{peak} \;\propto\; 2\zeta^{c}_{\mathrm{ref}}\,\omega_k J_k \quad (\text{no } \zeta), \qquad \text{decay} \;\propto\; e^{-\zeta_k\omega_k t} \quad (\zeta \text{ alone}), \tag{16}$$

with J_k the modal impulse the hammer delivers. Measured on the default plate over the full range of each knob, cascade and nonlinearity off:

	peak, $\sqrt{\zeta}$ rule	peak, $1/\zeta$ rule	decay
Viscous, $10^{-5} \to 10^{-2}$	+22.3 dB	0.5 dB	> 8 s $\to 0.68$ s
Material, $10^{-6} \to 10^{-3}$	+14.2 dB	1.2 dB	unchanged

The decay times are identical under both rules; only the level moved. The clamp on (15) spans $[10^{-3}, 10^{3}]$ and does not bind anywhere in the parameter ranges the plugin offers, which matters: a clamp that bit would reintroduce exactly the damping-dependent level the rule exists to remove. ζ^{c}_{ref} carries no physics, only gain staging, and is set so the default patch lands within 0.2 dB of the level it had before.

It is deliberately *not* the $\zeta_{\mathrm{ref}} = 10^{-2}$ that normalises the modal reconstruction $q_k = y_k/(2\zeta_k\omega_k^2)$ used by the Berger driver, the cascade source and the display field. Those are calibrated against it — g_{nl} in particular is a measured number that assumes it — so sharing one constant between the two roles would mean that every future adjustment of the output level silently rescaled the nonlinearity by its square.

Retunes triggered by parameter changes rewrite only the biquad coefficients (the states are preserved), which is inaudible for the small throttled steps used here.

4.4 Playing the plate: notes, channels and glide

A MIDI note can do two things — *strike* the plate and *tune* it — and these are two mechanisms rather than one, routed independently by channel. Both are needed, because eight sources mapped to their own notes would otherwise fight over the tuning: every note struck would also transpose the instrument.

- *Sources channel* (*Src Chan*, omni by default). A note-on strikes every source that is switched on and whose *note* parameter matches, each at its own position, hammer duration, force and spread, scaled by the note velocity. Sources may share a note number, in which case one key strikes the plate at several points at once (a chord on one plate, rather than a chord of plates).

- *Unmapped notes.* A note on that channel that no source claims strikes the last touched position when *Unmapped Hit* is on (the default, so that a keyboard plays the plate before anything has been mapped), and does nothing at all when it is off, which is what one wants once every source has its note.

- *Frequency channel* (*Freq Chan*, off by default). A note-on retunes the bank, $n \mapsto f_1 = 440 \cdot 2^{(n-69)/12}$, and strikes nothing.

- *Learning.* A source can be armed from its panel, in which case the next note-on on any channel is captured as that source's note and swallowed rather than played: a mapping that began with a hit nobody asked for would fire on the very note being used to teach it. The channel is deliberately not part of what is learned; routing is what the two channel controls decide, and folding it into the mapping would give the same note two meanings.

Setting the two channel controls equal plays and tunes from one keyboard; splitting them lets a second controller tune while the first strikes; leaving Frequency off gives a fixed-pitch plate struck from the keys.

What a played note sets is the f_1 of (11), so the whole spectrum transposes rigidly and the timbre is unchanged — unlike the nonlinear glide of §5.1, which moves T and so compresses the overtone ratios as it rises. The two are independent and superpose.

Retuning is a portamento, taken in the log domain and on the same decimated clock as the Berger update:

$$\log_2 f_1 \;\leftarrow\; \log_2 f_1 + \delta, \qquad \delta \;=\; \frac{\log_2 f_1^{\text{target}} - \log_2 f_1}{T_g f_s / P}, \tag{17}$$

with T_g the *Glide* time (0.1–100 ms) and $P = 32$ samples the update period. Two properties follow from the form of (17) rather than from tuning: a constant step in $\log_2 f$ makes the glide last T_g whatever the interval, and sizing the step from the total number of updates the glide spans, $T_g f_s/P$, rather than fixing it per update, makes it last T_g at any sample rate. The first note of a session lands directly, there being no previous pitch to glide from and no reason to put a swoop on a note nobody asked to bend. Notes are consumed in step with the render loop rather than at the block boundary, so a note lands on its own sample; the difference is audible on short blocks with a fast glide. Note-offs are ignored throughout: a struck plate rings out.

The bank is rewritten once the tuning has drifted about 2 cents from the pitch it currently sounds at — the same throttle, and the same state-preserving coefficient rewrite, as the Berger glide (§5.1) — so a portamento costs a bounded number of coefficient updates rather than one per sample.

Because there is a single bank (§4.3), tuning is global: a glide carries the whole ringing spectrum with it, including notes struck earlier that are still sounding. This is the behaviour of one plate being retuned, which is what the model is, and not that of a polyphonic instrument in which each note would hold its own pitch.

5 Nonlinear approximation

Large-amplitude plate vibration is governed by the von Kármán equations [25, 26]: transverse deflection stretches the mid-plane, producing an amplitude-dependent membrane stress that (i) raises all frequencies (the gong/tom *pitch glide*) and (ii) couples the modes, cascading energy towards high frequencies (the cymbal *crash*, up to wave turbulence [40]). Exact modal von Kármán synthesis requires the cubic coupling tensors Γ_{klmn} — $O(N^3)$–$O(N^4)$ storage and work [30, 39] — which is beyond a realtime budget at a few hundred modes. ModalDish implements two lightweight surrogates, one for each phenomenon; §5.5 compares them with the direct simulation of the full system, which is affordable in real time on a rectangle.

5.1 Dynamic tension: the Berger approximation

Berger [27] observed that for many plate problems the von Kármán membrane coupling is well approximated by a *spatially uniform* tension proportional to the integrated stretching:

$$\ddot{w} + \Delta^2 w - \Big(T_{\text{knob}} + \underbrace{\kappa \int_\Omega |\nabla w|^2 \mathrm{d}A}_{T_{\text{dyn}}}\Big)\Delta w = f. \tag{18}$$

In modal coordinates $\int |\nabla w|^2 = \sum_{kl} q_k q_l\, \boldsymbol{\varphi}_k^{\mathsf{T}} G \boldsymbol{\varphi}_l \approx \sum_k g_k q_k^2$: the stretching integral is measured by the very same G and g_k of §3.2, and the resulting uniform T_{dyn} feeds straight into the retune law (9). The Berger model captures the hardening (glide-up-and-relax) behaviour of plates and membranes and is a standard efficient approximation in the nonlinear vibration literature

[27, 28]; its perceptual signature on percussion is documented by Legge & Fletcher and Rossing [29, 24].

Amplitude estimation and calibration. The stretching integral needs no field reconstruction: the modes are mass-orthonormal, so $\int_\Omega w^2\,\mathrm{d}A = \sum_k q_k^2$ exactly and $\int_\Omega |\nabla w|^2 \mathrm{d}A \approx \sum_k g_k q_k^2$, one multiply-add per mode and per sample. What the filter bank holds is not q_k, however. A 0 dB-peak band-pass fed the true modal force is exactly $y_k = 2\zeta_k\omega_k\,\dot{q}_k$ — a *velocity*, normalised — so

$$q_k \;=\; \frac{y_k}{2\zeta_k\omega_k^2}, \qquad \gamma \;=\; \min\Big(\gamma_{\max},\; \alpha\cdot\mathrm{nonlin}\cdot\Big\langle\sum_k W_k\,y_k^2\Big\rangle\Big), \qquad W_k = \frac{g_k}{g_1}\left(\frac{\zeta_{\mathrm{ref}}}{\zeta_k}\right)^2\frac{1}{\nu_k^4}, \quad (19)$$

with $T_{\mathrm{dyn}} = \gamma\,\omega_1^2(T_{\mathrm{knob}})/g_1$, $\alpha = 500$, $\gamma_{\max} = 4$, and $\langle\cdot\rangle$ a 20-ms one-pole envelope. The missing $1/(4\zeta^2\omega^4)$ — ten decades across the bank — is why a naive $\sum_k g_k\langle y_k^2\rangle$ underestimates the stretching by orders of magnitude. Being derived from the velocity, each mode is a quarter period ahead of its true displacement; the cross terms $k \neq l$ and that phase both average out in the envelope.

By construction $\omega_1^2 \mapsto \omega_1^2\,(1+\gamma)$ exactly, and every other mode follows its own physical sensitivity g_k: γ is plate-independent ($\gamma = 0.4$ always means "mode 1 up ≈ 3 semitones"). It is also *damping*-independent by construction: after an impulse $y_k \sim 2\zeta_k\omega_k$, so ζ_k cancels in $W_k y_k^2$ and the peak driver varies by less than 12% over $\zeta_v, \zeta_m \in [10^{-5}, 10^{-3}]$ on the default plate. And being a quantity integrated over Ω, it is the same wherever one listens: the glide belongs to the plate, not to the pickup.

Realtime scheduling. γ is updated every 32 samples and slewed towards its target with a 3 ms time constant — a factor 0.2 per update at 48 kHz, the rate it was voiced at, with the coefficient recomputed in `prepare` so that the smoothing lasts the same wall-clock time at any sample rate. The biquads are rewritten only when mode 1 would move by more than ≈ 2 cents ($|\Delta\gamma| > 0.0024\,(1+\gamma)$, since $\Delta\omega/\omega = \frac{1}{2}\Delta\gamma/(1+\gamma)$). Coefficient rewrites preserve filter states; the throttling keeps them free of zipper artefacts, following the same strategy as MechanOdd's gate-morphed damping.

5.2 Mode cascade: a multi-band cubic ladder

A memoryless cubic of a signal band-limited to $[0, B_\omega]$ produces inter-modulation products only up to $3B_\omega$, concentrated near the original band. A single "distort the low modes, inject into the high ones" pass therefore *cannot* reach the top of the bank: with sources below $\sim 12 f_1$ the products die out by ~ 3 kHz while the bank extends to 9 kHz, and all that remains audible of such a scheme is the side effect of its overlap floor — high-frequency damping. The physical cascade climbs the spectrum *iteratively*: products beget products, stage after stage [40].

ModalDish reproduces this with a *windowed* band ladder. The active modes are split into $N_b = 8$ contiguous index bands $\mathcal{B}_0 \ldots \mathcal{B}_7$ (equal index ranges are equal frequency ranges, plate modal density being constant); band b is driven by the cubic of the W bands directly below it only (default $W = 4$),

$$w^{(b)}(t) = \frac{S}{\sqrt{|\Omega|}}\sum_{c=b-W}^{b-1}\frac{1}{n_c}\sum_{k\in\mathcal{B}_c} v_k\,y_k(t), \qquad F_b(t) \;=\; A\,g_b(t)\,\tanh^3\!\big(B\,w^{(b)}(t)\big), \quad (20)$$

fed to mode $l \in \mathcal{B}_b$ with weight $\varphi_l(\boldsymbol{x}_h)/c_l$ (dividing by the bandwidth compensation c_l, which is re-applied on output, makes the audible cascade level independent of the damping settings). The source is the plate's *motion*, over the whole plate: $v_k = \zeta_{\mathrm{ref}}/(\zeta_k\nu_k)$ is the modal velocity

scale, $n_c = \sum_{k\in\mathcal{B}_c} 1/\nu_k$ normalises each band, $1/\sqrt{|\Omega|}$ is what a mass-normalised φ_k is typically worth, and $S = 3$ is the one global gain (§5.2). The source *window* matters: summing all lower bands would let the loud strike band drive the top bands quasi-directly and the whole spectrum lights up within milliseconds; with the window, each stage extends the spectral reach by at most a factor 3 and energy genuinely climbs rung by rung. The per-band gate $g_b(t) \in [0,1]$ adds *transfer inertia*: it follows the source presence through an attack/release envelope whose attack time grows with the band's height on the ladder ($\tau_b = 30\,\mathrm{ms} \times b$ and a 2 s release in the voiced defaults), so the brightening is a progressive glow and the pumping tails off smoothly instead of collapsing with the cube of the decaying ring. Voicing found a hard-saturating transfer most convincing; the shipped combination — an injection gain pinned high and a moderate drive — is derived in §5.2. A cubic is the leading nonlinearity of von Kármán dynamics [30]; at low amplitude (20) vanishes as w^3 (the linear model is recovered exactly), and at high amplitude the bands brighten in sequence, reproducing the amplitude-gated crash of cymbals and gongs [24]. The cascade internals that remain adjustable (B, the source window, the gate times, the overlap floor below and the depletion) are exposed as plugin parameters; the values quoted here are the defaults, A being fixed (§5.2). Note also that the material damping law $\zeta \propto \nu$ dominates the high-mode decay when hunting long shimmer: at $\zeta_m = 10^{-4}$ a mode at $\nu = 80$ decays in milliseconds whatever the cascade does.

Modal-overlap floor. The cubic's inter-modulation products are broadband, but at typical plate dampings the receiving resonances are needles: bandwidth $2\zeta_l f_l$ of a few hertz against a mean mode spacing $\Delta f \approx 0.64 f_1$ (the plate Weyl law gives a constant modal density $\mathrm{d}N/\mathrm{d}\nu = \pi/2$ for the unit square), i.e. a modal overlap of order 0.1. Most of the injected energy then falls *between* modes and is rejected — heard as distortion of isolated partials rather than a cascade, whereas the crash regime of real cymbals has overlap $\gg 1$ [24]. The target bandwidths are therefore floored, proportionally to the cascade control, at an adjustable fraction (*Overlap*, default 0.1) of the *local* mode spacing (measured on the actual f_k array),

$$\zeta_l \;\geq\; \text{cascade} \times \text{Overlap} \times \frac{\frac{1}{2}(f_{l+1} - f_{l-1})}{2 f_l}, \qquad l \notin \mathcal{B}_0, \tag{21}$$

turning the receiving comb into a quasi-continuum when the cascade is engaged (and, physically consistently, shortening the high modes' ring — nonlinear energy spreading is itself a loss channel for individual modes). At cascade = 0 the linear model is untouched.

Why the source is the motion and not the sound. The obvious source signal for (20) is the one already being summed for the output, $\varphi_k(\boldsymbol{x}_o)\, c_k\, y_k$ — and that is precisely what it must not be. Two things stand between the audible signal and the plate's motion, and the ladder's cubic would raise both to the third power.

The first is the output gain (15). By (16) the bank models plate *acceleration*, so the audible signal carries a frequency tilt ω_k that the plate's motion does not. Weighting the source by $v_k \propto 1/(\zeta_k \nu_k)$ removes both that tilt and the bandwidth factor, since $y_k \propto \zeta_k \omega_k$; what is left is the modal velocity.

The per-band divisor n_c then supplies the *frequency* balance the ladder needs: physical modal velocities fall steeply with frequency, and a raw velocity sum outweighs band 1 with band 0 by a factor ~ 900, starving the upper rungs. n_c is fixed by the mode indices alone, so no damping re-enters.

The second is the pickup. A mode nodal at $\boldsymbol{x}_o$ contributes nothing to the audible signal however hard the plate is moving there, and a real plate's cubic coupling does not know where the microphone is. The source weight is therefore the constant $1/\sqrt{|\Omega|}$, the typical size of a mass-normalised mode shape ($\int_\Omega \varphi_k^2 \mathrm{d}A = 1$), which holds the drive's level on any geometry while tying it to no listening position: across five pickup positions the high-mode motion the

cascade creates is identical to the last digit. What the pickup changes is what one *hears* of that motion, which is what a pickup is for. The *injection* does keep $\varphi_l(\boldsymbol{x}_h)$: the hit point is where the plate deflects most, the one position the nonlinearity has a physical claim to, and it is what ties the cascade's character to how the plate is played.

The Cascade knob therefore acts directly, with no damping correction on it. `Tests/CascadeMeasure.cpp` renders one strike through the real synthesis and asserts what the construction promises — the drive is damping-free, so any real spread would mean a damping term had crept back into it:

Shimmer at 300 ms, spread over	dB
Viscous $\zeta_v \in [10^{-5}, 10^{-3}]$	5.0
Material $\zeta_m \in [7 \cdot 10^{-6}, 3 \cdot 10^{-4}]$	6.6
Output point, five positions (drive, measured on the modes)	0.00

The few dB left on the damping rows are not drive: the targets ring for longer or shorter with the damping, and the middle rungs are sources for the upper ones. Meanwhile the Cascade knob spans 35 dB over its travel and a force-10 hit shimmers 75 dB more than a force-1 one — the effect stays amplitude-gated, which is the part that *is* physical.

The ladder has two controls. The injection gain A is pinned at 10: a gain and an amount in series are one control between them, so the Cascade amount now spans that range on its own. What is left beside it is Drive, which moves the tanh knee and so changes the *shape* of the carrier rather than its size — the one of the three that was not a duplicate. Two consequences are worth stating. Pinning A high puts the ladder on the steep part of $\tanh^3$, which makes the shimmer more sensitive to anything altering the source level: the residual spread across the Material knob, 25 dB before, is 57 dB now. It also made the amount knob well behaved, the run above rising monotonically where it previously turned back on itself near the top.

The amount multiplies the bandwidth floor of (21) as well as the injection. That is not tidiness: a floor that ignored the amount would widen — and so damp — every target mode of a plate whose cascade was switched off, making a control that was doing nothing else audibly shorten the top of the spectrum.

Depletion (low-frequency loss). The transfer (20) is additive: it copies energy upward but takes none from the sources, so the struck low modes ring on untouched under the shimmer — audibly two disconnected layers. In the real cascade the low modes *lose* the energy they hand upward. ModalDish models this as an extra dissipation on the source bands: each sample, the filter states of the modes in band b are scaled by

$$d_b \;=\; 1 - \text{Deplete} \times \text{cascade} \times \varepsilon\, \bar{g}_b^{\uparrow}, \qquad \varepsilon = 1 - e^{-1/(\tau_d f_s)}, \quad \tau_d = 69.4\,\text{ms} \tag{22}$$

(Deplete = 0.07 by default), where $\bar{g}_b^{\uparrow}$ is the mean gate level of the bands fed by band b: the lows decay exactly when, and as strongly as, they are pumping the highs, gluing the two ends of the spectrum. The loss is stated as a time constant rather than as a per-sample fraction so that a given *Deplete* setting removes energy at the same rate whatever the sample rate; $\varepsilon = 3 \cdot 10^{-4}$ at 48 kHz, the value it was voiced at. The mechanism is purely dissipative (it only removes energy), so it cannot affect stability.

Stability by structure. The obvious way to spread energy upward is a feedback loop closed through the bank's own output, and it does not survive contact with the gains involved. Around mode k's resonance such a loop transmits with $|\varphi_k(\boldsymbol{x}_h)| \cdot |\varphi_k(\boldsymbol{x}_o)| \cdot c_k$, and the compensation $c_k \le 32$ must not be forgotten: an unnormalised loop is super-critical and latches into a full-scale limit cycle the moment the cubic's slope wakes up. Bounding the loop slope by the small-gain theorem [43, 44] restores stability but is so conservative (the bound assumes worst-case coherent

feedback at the single worst mode) that the effect becomes inaudible. The ladder (20) dissolves the dilemma structurally: band b receives only from bands $c < b$ and contributes only to bands $c > b$, so the transfer graph over bands is a strict DAG — no loop exists at any stage, stability is unconditional, and the injection gain A is free to be set for audibility ($A = 10 \times$ cascade, the $\tanh^3$ keeping every stage bounded).

5.3 The statistical mode tail

Even a 256-mode FEM bank remains a sparse comb at the top of the spectrum, and solving for many hundreds of accurate FEM modes is prohibitive (§3) — while a convincing crash needs a dense receiving continuum up to Nyquist. ModalDish therefore fills the bank (to 1024 modes in all) with *synthetic* modes above the FEM set, built from asymptotics instead of the discrete operator:

- *Frequencies*: the plate Weyl law gives a constant modal density in ω; the tail continues at the spacing $\Delta\omega$ measured on the top half of the FEM spectrum, with gaps jittered by $\pm 40\%$ (deterministically seeded) to avoid periodicity artefacts.
- *Tension sensitivities*: for an asymptotic mode of wavenumber κ, $\lambda = \kappa^4$ and $g = \kappa^2$, hence $g = \sqrt{\lambda}$ exactly (verified by the simply supported square, where $g_{mn} = \pi^2(m^2{+}n^2) = \sqrt{\lambda_{mn}}$) — so the tail obeys the tension law (9) and the Berger glide like any FEM mode.
- *Shapes*: high-frequency eigenfunctions of irregular domains behave like superpositions of random plane waves (Berry's random-wave model [41]); each tail mode carries one, $\varphi(\boldsymbol{x}) = \sqrt{2/A}\,\cos(\boldsymbol{\kappa}\cdot\boldsymbol{x} + \psi)$ with $|\boldsymbol{\kappa}| = \lambda^{1/4}$, random direction and phase, and the $\sqrt{2/A}$ prefactor reproducing the mass normalisation ($\overline{\varphi^2} = 1/A$). Hit and pickup positions therefore still modulate the tail response.

The tail modes enter the synthesis exactly like FEM modes (hammer excitation, damping law, cascade bands, dynamic tension); only the contour display is FEM-only, since tail modes have no mesh shape. The identity-defining low modes stay exact, and the top of the bank becomes the quasi-continuum the cascade ladder needs — the reasoning is that of statistical energy analysis [42]: above a few dozen mode indices, individual eigenfrequencies are perceptually interchangeable and only their density and statistics matter.

5.4 Stereo placement of the tail

The random-wave shapes that make the tail convincing also make it *mono*. A pickup at $\boldsymbol{x}_p$ reads mode k with gain $\varphi_k(\boldsymbol{x}_p)$, so the power it collects from a group of N modes is

$$P(\boldsymbol{x}_p) \;=\; \sum_k \varphi_k(\boldsymbol{x}_p)^2 \;\xrightarrow[N\to\infty]{}\; \frac{N}{A}, \tag{23}$$

independently of $\boldsymbol{x}_p$, since mass normalisation fixes $\overline{\varphi^2} = 1/A$ at every interior point (§5.3). Every term is positive, so the sum concentrates: measured on the default plate, two pickups differ by 2.1 dB over the first four modes and by 0.29 dB over 512 of them. The cascade regenerates its shimmer across hundreds of tail modes at once, so (23) applies in full and the whole of it arrives at both channels at the same level. The low modes, being few, keep the strong per-mode differences ($\approx$ 15 dB rms between two points, more near a nodal line) that give the bottom of the spectrum its width.

No amount of decorrelation repairs this, because nothing is correlated: the interchannel correlation of the fine structure above 2 kHz is already only -0.2 to -0.35. What matches is the *envelope* (0.96 to 0.98), and it matches at every lag within ± 1 ms, so a delay — including

the physically motivated one, the bending-wave travel time between pickups, which is 200 to 560 μs here and squarely in the interaural range — can only translate the correlation peak, never lower it. It would move the shimmer off centre, not widen it.

What does not concentrate is the *signed* sum over a band,

$$S_b(\boldsymbol{x}_p) \;=\; \sum_{k\in b} \varphi_k(\boldsymbol{x}_p), \qquad \mathbb{E}[S_b] = 0, \quad \mathrm{Var}[S_b] = \frac{N_b}{A}, \tag{24}$$

a random walk whose terms carry Berry's random phases and so cancel rather than accumulate. $|S_b|$ stays of order $\sqrt{N_b/A}$ for any N_b, and the ratio $|S_b(\boldsymbol{x}_1)|/|S_b(\boldsymbol{x}_2)|$ between two pickups therefore keeps a broad distribution however many modes the band holds. ModalDish uses it as the band's stereo placement,

$$p_b \;=\; \frac{|S_b^R| - |S_b^L|}{|S_b^R| + |S_b^L|} \;-\; \bar{p}, \tag{25}$$

clamped to $|p_b| \le 0.8$ and fed to the same equal-power law as the pickup pan, then folded into the per-mode output weights so that the audio loop is untouched and the whole construction costs nothing per sample. Subtracting the mean $\bar{p}$ keeps (23), which is a real property of the plate: the two channels genuinely do receive the same total, and only the placement of one band relative to its neighbours is the random walk's to decide. Without it the tail acquires a lateral shift of several dB, which is the one thing a delay could already have done.

Bands are spaced geometrically, about three to the octave. The bound that matters is perceptual rather than numerical: bands much narrower than an auditory filter are summed back together by the listener, which re-centres them, and a third of an octave stays comfortably wider than an ERB across the tail's range. Measured one auditory filter at a time on the default plate, with two pickups hard panned at different points, the rms interchannel level difference across the tail rises from 1.1–1.8 dB to 5.1–7.9 dB depending on mode count, with the mono sum unchanged to 0.001 dB and a single centred pickup left untouched.

Only the tail is treated this way. Below the last FEM mode the shapes are computed eigenvectors carrying a genuine position dependence, and replacing them with a band statistic would discard the very thing §3 went to the trouble of solving for.

5.5 Relation to other nonlinear plate synthesis

The closest published work to ModalDish's objective is the real-time gong synthesis of Bilbao, Webb, Wang and Ducceschi [31], which reaches the same sounds — pitch glide, crash, swell — from the opposite direction, and the contrast is worth stating plainly because it delimits what the surrogates of §5.1 and §5.2 can and cannot claim.

Direct time-domain simulation of the von Kármán plate for synthesis goes back to Bilbao's offline work [32, 39]; what [31] adds is the algorithmic work needed to bring it inside a real-time budget.

That work simulates the dynamic Föppl–von Kármán system directly: the deflection w and the Airy stress function Φ are stepped on a uniform Cartesian grid, with the nonlinearity made fully explicit by a scalar auxiliary variable (energy quadratisation), a rank-one system solved in closed form by Sherman–Morrison, and a discrete energy balance that yields a provable stability condition [33, 31]. The nonlinear effects are not modelled separately: they are consequences of the one equation, so the glides come out non-uniform across partials and the crash appears on its own as the amplitude rises. The price is paid in three places:

- *Geometry.* The domain is a rectangle with simply supported conditions on both w and Φ. This is not incidental. The run-time bottleneck is the biharmonic solve $\Delta\Delta\Phi = -L(w, w)$ needed at every sample, which consumes 72–76% of the time step, and the fast solver that

makes it affordable [34] exploits the Toeplitz-block-Toeplitz structure that the rectangle and those boundary conditions produce. An arbitrary outline would cost the solver, and with it the real-time budget.

- *Cost.* Reported timings are 0.15–0.57 s of computation per second of audio on a recent laptop CPU, and 0.26–0.99 s on an older one, for grids of roughly 300 to 720 interior points. ModalDish's full bank — 256 modes with both the Berger glide and the cascade engaged — measures 0.054 s per second of audio on an Intel i7-10810U, roughly an order of magnitude less, because nothing is solved at run time: the eigenproblem was solved once, offline (§3; 0.5 s for the default plate), and what remains is a bank of biquads whose coefficients are rewritten when the tension moves.

- *Resolution coupling.* The grid spacing is tied to the sample rate by the stability condition, so spatial resolution is not a free modelling choice. In a modal scheme the mesh density and the audio rate are independent.

What is given up in exchange is exactness, and that paper names the limitation precisely: Berger's approximation "[is] able to replicate pitch glides, but not the energy cascade to high frequencies characteristic of crashes" [31]. That is correct, and it is why §5.2 exists as a *separate* mechanism rather than falling out of §5.1. Two consequences follow. First, the glide here is driven by a single scalar T_{dyn}: every mode follows it through its own g_k, so the pattern of glides is fixed by geometry and boundary conditions, where in a von Kármán solution mode k's frequency shift depends on which modes actually carry the energy and therefore evolves through the decay. Second, the cascade of §5.2 is a caricature of the coupling, not the coupling: it reproduces the phenomenon — energy climbing the spectrum rung by rung, gated by amplitude — with a band ladder whose gains are voiced by ear, and it carries no energy balance. Stability comes from the transfer graph being acyclic rather than from a numerical invariant.

The exact modal route sits between the two. Projecting von Kármán onto the linear modes gives cubic coupling tensors Γ_{klmn}, which is the basis of the gong and cymbal synthesis of Ducceschi and Touzé [30] and of the analytical work on circular plates and shallow shells by Touzé, Thomas and Chaigne [35, 36]; Nguyen and Touzé extend it to plates of variable thickness for cymbals [37]. That route keeps the modal picture — and would keep ModalDish's arbitrary geometry, since the tensors can in principle be assembled from any set of FEM modes — but the $O(N^3)$–$O(N^4)$ tensor is what rules it out at $N = 256$ in a plugin, and it is the reason the two cheap surrogates were chosen instead.

Finally, the mechanism of §5.1 is not new in synthesis: an amplitude-dependent tension computed from a global stretching estimate, fed back into the linear model's tuning, is the plate analogue of the tension-modulation models used for plucked strings, where the same first-order retuning reproduces the pitch drop of a hard pluck [38]. What is specific here is that the tension enters through the FEM sensitivities g_k of §3.2, so it costs one scalar per audio block on any drawn shape.

6 Implementation summary

Stage	Thread	Notes
Shape / boundary editing	message	`ShapeCanvas`; outline + arc-parameterised segments
Mesh generation	message	milliseconds; preview in `FemViewComponent`
Eigensolution (6)	background	`BackgroundTaskRunner`; progress; seconds
Model publication	message → audio	atomic pointer, generation-acknowledged reclamation
Note routing §4.4	audio	per-channel; strike and tune are separate calls
Filter bank (10)	audio	allocation-free; one bank, retunes throttled
Nonlinearities (19), (20)	audio	decimated γ update; bounded feedback

The numerical core is pure C++17 with no JUCE dependency and is validated by a console test suite; it lives in the `FxmeTools` submodule, split between `FxmeTools/acoustics` (`FemMesh`, `PlateModes`: the plate physics) and `FxmeTools/math` (matrix storage, Cholesky, subspace eigensolver: the part that has never heard of a plate), so that either half is reusable on its own.

References

[1] A. W. Leissa, *Vibration of Plates*, NASA SP-160, Washington DC, 1969.

[2] S. Timoshenko, S. Woinowsky-Krieger, *Theory of Plates and Shells*, 2nd ed., McGraw-Hill, 1959.

[3] K.-J. Bathe, *Finite Element Procedures*, 2nd ed., Prentice Hall / K.-J. Bathe, 2014.

[4] E. Catmull, R. Rom, "A class of local interpolating splines", in *Computer Aided Geometric Design*, Academic Press, 1974, pp. 317–326.

[5] A. Bowyer, "Computing Dirichlet tessellations", *The Computer Journal* 24(2), 1981, 162–166.

[6] D. F. Watson, "Computing the n-dimensional Delaunay tessellation with application to Voronoï polytopes", *The Computer Journal* 24(2), 1981, 167–172.

[7] L. S. D. Morley, "The triangular equilibrium element in the solution of plate bending problems", *Aeronautical Quarterly* 19, 1968, 149–169.

[8] R. Rannacher, "Nonconforming finite element methods for eigenvalue problems in linear plate theory", *Numerische Mathematik* 33, 1979, 23–42.

[9] I. Babuška, J. Osborn, "Eigenvalue problems", in *Handbook of Numerical Analysis*, Vol. II, North-Holland, 1991, 641–787.

[10] E. Cuthill, J. McKee, "Reducing the bandwidth of sparse symmetric matrices", in *Proc. 24th National Conference of the ACM*, 1969, pp. 157–172.

[11] A. George, J. W. H. Liu, *Computer Solution of Large Sparse Positive Definite Systems*, Prentice-Hall, 1981.

[12] D. A. Dunavant, "High degree efficient symmetrical Gaussian quadrature rules for the triangle", *International Journal for Numerical Methods in Engineering* 21, 1985, 1129–1148.

[13] K.-J. Bathe, E. L. Wilson, "Solution methods for eigenvalue problems in structural mechanics", *International Journal for Numerical Methods in Engineering* 6, 1973, 213–226.

[14] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.

[15] R. Courant, D. Hilbert, *Methods of Mathematical Physics*, Vol. I, Interscience, 1953.

[16] T. Kato, *Perturbation Theory for Linear Operators*, 2nd ed., Springer, 1976.

[17] R. L. Fox, M. P. Kapoor, "Rates of change of eigenvalues and eigenvectors", *AIAA Journal* 6(12), 1968, 2426–2429.

[18] J.-M. Adrien, "The missing link: modal synthesis", in G. De Poli, A. Piccialli, C. Roads (eds.), *Representations of Musical Signals*, MIT Press, 1991, 269–298.

[19] J. D. Morrison, J.-M. Adrien, "MOSAIC: a framework for modal synthesis", *Computer Music Journal* 17(1), 1993, 45–56.

[20] K. van den Doel, D. K. Pai, "Modal synthesis for vibrating objects", in K. Greenebaum (ed.), *Audio Anecdotes III*, A. K. Peters, 2003.

[21] S. Bilbao, *Numerical Sound Synthesis: Finite Difference Schemes and Simulation in Musical Acoustics*, Wiley, 2009.

[22] R. Bristow-Johnson, "Cookbook formulae for audio EQ biquad filter coefficients" (the "Audio EQ Cookbook"), https://www.w3.org/TR/audio-eq-cookbook/.

[23] A. Chaigne, C. Lambourg, "Time-domain simulation of damped impacted plates. I. Theory and experiments", *Journal of the Acoustical Society of America* 109(4), 2001, 1422–1432.

[24] N. H. Fletcher, T. D. Rossing, *The Physics of Musical Instruments*, 2nd ed., Springer, 1998.

[25] T. von Kármán, "Festigkeitsprobleme im Maschinenbau", *Encyklopädie der mathematischen Wissenschaften* IV/4, 1910, 311–385.

[26] H.-N. Chu, G. Herrmann, "Influence of large amplitudes on free flexural vibrations of rectangular elastic plates", *Journal of Applied Mechanics* 23, 1956, 532–540.

[27] H. M. Berger, "A new approach to the analysis of large deflections of plates", *Journal of Applied Mechanics* 22, 1955, 465–472.

[28] A. H. Nayfeh, D. T. Mook, *Nonlinear Oscillations*, Wiley, 1979.

[29] K. A. Legge, N. H. Fletcher, "Nonlinearity, chaos, and the sound of shallow gongs", *Journal of the Acoustical Society of America* 86(6), 1989, 2439–2443.

[30] M. Ducceschi, C. Touzé, "Modal approach for nonlinear vibrations of damped impacted plates: application to sound synthesis of gongs and cymbals", *Journal of Sound and Vibration* 344, 2015, 313–331.

[31] S. Bilbao, C. Webb, Z. Wang, M. Ducceschi, "Real-time gong synthesis", in *Proceedings of the 26th International Conference on Digital Audio Effects (DAFx23)*, Copenhagen, 2023, pp. 1–8.

[32] S. Bilbao, "Sound synthesis for nonlinear plates", in *Proceedings of the 8th International Conference on Digital Audio Effects (DAFx-05)*, Madrid, 2005, pp. 243–248.

[33] S. Bilbao, M. Ducceschi, F. Zama, "Explicit exactly energy-conserving methods for Hamiltonian systems", *Journal of Computational Physics*, 2023.

[34] Z. Wang, M. Puckette, "A fast algorithm for the inversion of the biharmonic in plate dynamics applications", in *Proceedings of the 5th Stockholm Music Acoustics Conference*, Stockholm, 2023.

[35] C. Touzé, O. Thomas, A. Chaigne, "Asymmetric non-linear forced vibrations of free-edge circular plates, part I: theory", *Journal of Sound and Vibration* 258(4), 2002, 649–676.

[36] O. Thomas, C. Touzé, A. Chaigne, "Non-linear vibrations of free-edge thin spherical shells: modal interaction rules and 1:1:2 internal resonance", *International Journal of Solids and Structures* 42, 2005, 3339–3373.

[37] Q. Nguyen, C. Touzé, "Nonlinear vibrations of thin plates with variable thickness: application to sound synthesis of cymbals", *Journal of the Acoustical Society of America* 145, 2019, 977–988.

[38] T. Tolonen, V. Välimäki, M. Karjalainen, "Modeling of tension modulation nonlinearity in plucked strings", *IEEE Transactions on Speech and Audio Processing* 8(3), 2000, 300–310.

[39] S. Bilbao, "A family of conservative finite difference schemes for the dynamical von Kármán plate equations", *Numerical Methods for Partial Differential Equations* 24(1), 2008, 193–216.

[40] C. Touzé, S. Bilbao, O. Cadot, "Transition scenario to turbulence in thin vibrating plates", *Journal of Sound and Vibration* 331(2), 2012, 412–433.

[41] M. V. Berry, "Regular and irregular semiclassical wavefunctions", *Journal of Physics A: Mathematical and General* 10(12), 1977, 2083–2091.

[42] R. H. Lyon, R. G. DeJong, *Theory and Application of Statistical Energy Analysis*, 2nd ed., Butterworth-Heinemann, 1995.

[43] G. Zames, "On the input-output stability of time-varying nonlinear feedback systems — Part I", *IEEE Transactions on Automatic Control* 11(2), 1966, 228–238.

[44] H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, 2002.